

Интерфейс бронирования переговорной комнаты

Задача

Реализовать интерфейс бронирования переговорной комнаты в течение рабочего дня.

Рабочий день: **09:00–18:00**.

Пользователь должен иметь возможность:

- выбрать дату;
- посмотреть существующие бронирования на выбранную дату;
- выбрать время начала и окончания;
- создать бронирование;
- редактировать бронирование;
- удалить бронирование.

Формат отображения — на ваш выбор: список, таймлайн, сетка слотов или календарь дня.

Бизнес-правила

1. Время начала и окончания — в пределах рабочего дня.
2. $\text{start} < \text{end}$.
3. Минимальная длительность — **30 минут**.
4. Максимальная длительность — **2 часа**.
5. Бронирования не могут пересекаться.
 - Касание границ конфликтом не считается: 10:00–11:00 и 11:00–12:00 — допустимо.
6. Нельзя бронировать прошедшее время.
 - Для сегодняшней даты прошедшие слоты недоступны.
 - Для прошедших дат создание брони недоступно.
7. При редактировании текущая бронь не должна конфликтовать сама с собой.
8. Сервер может вернуть конфликт, даже если UI считал слот свободным.
 - Это ожидаемое поведение, а не баг.
 - UI должен корректно его обработать: показать понятное сообщение, обновить список брони и не потерять данные, введённые пользователем.

Что важно показать

- Аккуратную работу с бизнес-логикой и валидацией.
- Корректные состояния UI: загрузка, пустое состояние, ошибка, отправка формы.
- Обработку серверных ошибок, в том числе конфликтов.
- Разделение слоя API и UI.
- Читаемый TypeScript и понятную структуру кода.
- Адаптивность и базовую доступность.

Дизайн не оценивается как художественная работа — оценивается инженерная аккуратность.

Стек

- React/ Next.js/Vue.js.
- TypeScript — обязательно.
- Стили, state-менеджмент, формы — на ваш выбор.
- UI-библиотеки и иконки использовать можно.

Реальный бэкэнд, авторизация, БД и интеграции не нужны.

Если нужен сервер — реализуйте mock API (MSW, Route Handlers, in-memory store и т.п.).

Сетевой слой должен быть отделён от UI и легко заменяем на реальный API.

Примерный контракт:

GET /api/bookings?date=YYYY-MM-DD

POST /api/bookings

PATCH /api/bookings/:id

DELETE /api/bookings/:id

```
type Booking = {  
  id: string;  
  date: string; // YYYY-MM-DD  
  start: string; // HH:mm  
  end: string; // HH:mm  
  title?: string;  
};
```

Сервер валидирует бизнес-правила и умеет возвращать конфликт (`409`).

Таймбокс

Ориентир: **4–6 часов**.

Если не успеваете — остановитесь и в README опишите, что бы доделали и какие были приоритеты.

Что сдать

- Ссылку на репозиторий.
- README: как запустить, что сделано, что не сделано, сколько времени заняло.
- Ссылку на deployed demo (Vercel, Netlify и т.п.)
- чат-переписка с AI

Если что-то кажется неоднозначным — задавайте вопросы.

Это лучше, чем молчаливые догадки.